



C++ CONTROL STATEMENTS

DSA Marathon Master Cheat Sheet — Placement & Technical Interview Guide

Comprehensive • Beginner Friendly • C++17 / C++20 Ready • Interview Focused

SECTION 1: Introduction to Control Statements

1. What are Control Statements?

In default C++ program execution, statements are executed sequentially, line by line, from top to bottom. Control statements are language constructs that alter this standard linear flow of execution based on specific conditions or decisions made at runtime.

2. Why Do We Use Control Statements?

- Decision Making: Execute specific blocks of code only when dynamic runtime conditions are met.
- Branching: Guide program execution through alternate computational paths based on logic or user input.
- Code Efficiency: Avoid redundant calculations and execute only the necessary code paths.
- Error Prevention: Validate inputs and guard against runtime bugs (e.g. preventing division by zero).

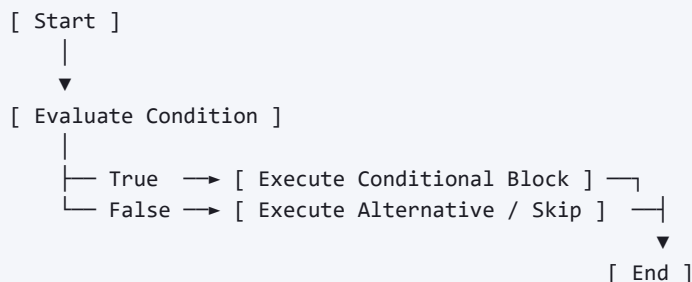
3. Types of Control Statements in C++

1. Selection / Decision Statements — if, if-else, else-if ladder, nested if-else, switch.
2. Iteration / Looping Statements — for, while, do-while.
3. Jump Statements — break, continue, goto, return.

4. Real-Life Example: ATM Withdrawal Logic

1. Condition 1: Is the entered PIN correct? If NO → Display "Invalid PIN".
2. Condition 2: Is requested amount ≤ Account Balance? If NO → Display "Insufficient Funds".
3. Action: If both are YES → Dispense Cash and Update Balance.

5. Flow of Execution Overview



6. Memory & Assembly Level Visualization

At the CPU level, decision-making evaluates conditional expressions into Boolean flag values (0 or 1) held in status registers. Conditional branch instructions like JZ (Jump if Zero / False) or JNZ (Jump if Not Zero / True) change the Instruction Pointer (IP) address, hopping to different memory regions that hold the target code blocks.

SECTION 2: Topic-by-Topic Comprehensive Coverage

1. The if Statement

Definition: The fundamental decision-making statement. It executes a block of code if, and only if, the tested expression evaluates to true (non-zero).

Why & When to Use: Use it when an action needs to happen conditionally, but no action is required when the condition fails.

Syntax

```
C++
if (condition) {
    // Statements executed if condition is true
}
```

Keyword `if` signals the conditional check. The condition is a boolean expression inside parentheses. The block `{ }` is the body executed only when that condition is true.

Step-by-Step Working & Dry Run

```
C++
#include <iostream>
using namespace std;
int main() {
    int number = 15;
    if (number > 10) {
        cout << "Number is greater than 10" << endl;
    }
    return 0;
}
```

Dry Run

Step	Variable State	Condition Check	Result	Action Taken
1	number = 15	15 > 10	true (1)	Enter block, execute cout.

Output: Number is greater than 10

Time Complexity: $O(1)$ **Space Complexity:** $O(1)$

★ INTERVIEW TIP

Non-zero numbers are treated as true in C++ — `if(-5)` will execute!

✓ BEST PRACTICES & PLACEMENT TIPS

Always use curly braces `{ }` even for single-line statements, to avoid accidental bugs during code refactoring.

2. The if-else Statement

Definition: A two-way selection statement that provides an alternative execution path when the condition evaluates to false.

Why & When to Use: Use it when you have exactly two mutually exclusive outcomes (e.g. Even/Odd, Pass/Fail, Eligible/Ineligible).

Syntax

```
C++
if (condition) {
    // Executed if condition is true
} else {
    // Executed if condition is false
}
```

Code Example & Dry Run

```
C++
#include <iostream>
using namespace std;
int main() {
    int age = 16;
    if (age >= 18) {
        cout << "Eligible to Vote" << endl;
    } else {
        cout << "Not Eligible to Vote" << endl;
    }
    return 0;
}
```

Dry Run

Step	Variable	Condition Check	Result	Action Taken
1	age = 16	16 >= 18	false	Bypass if block, enter else block.

Output: Not Eligible to Vote

Time Complexity: O(1) Space Complexity: O(1)

✗ COMMON MISTAKE

Writing an else statement without a preceding if, or placing a semicolon immediately after if(cond); which detaches the block that follows.

3. The else-if Ladder

Definition: A multi-way decision structure used to test several sequential conditions. Conditions are evaluated top to bottom until one evaluates to true.

Why & When to Use: Use it when you need to select one path among multiple non-overlapping conditions (e.g. a grading system, tax slabs).

Syntax

```
C++
if (condition1) {
    // Statements for condition1
} else if (condition2) {
    // Statements for condition2
} else {
    // Statements if all conditions are false
}
```

Code Example & Output

```
C++
#include <iostream>
using namespace std;
int main() {
    int marks = 78;
    if (marks >= 90) cout << "Grade A";
    else if (marks >= 75) cout << "Grade B";
    else if (marks >= 50) cout << "Grade C";
    else cout << "Fail";
    return 0;
}
```

Output: Grade B

Dry run: 78 >= 90 is false. Next, 78 >= 75 is true → prints "Grade B" and bypasses every remaining check.

4. Multiple if Statements

Definition: Multiple independent if statements placed one after another. Every single if condition is evaluated regardless of what the preceding ones resolved to.

When to Use: When multiple conditions can simultaneously be true and the resulting actions are independent of each other (e.g. applying several discount rules at once).

Comparison Code Example

```
C++
int val = 20;
// Multiple IF (all evaluated)
if (val > 10) cout << ">10 ";           // TRUE -> Prints
if (val > 15) cout << ">15 ";           // TRUE -> Prints

// Else-If Ladder (stops at first match)
if (val > 10) cout << ">10 ";           // TRUE -> Prints & Exits
else if (val > 15) cout << ">15 ";       // Skipped!
```

Output: Multiple IF Output: >10 >15 | Else-If Output: >10

5. Multiple Conditions (Logical Operators)

Definition: Combining several boolean sub-expressions into a single decision, using the logical operators &&, ||, and !.

Code Example: Range & Eligibility Check

C++

```
int age = 22;
bool hasLicense = true;
if (age >= 18 && hasLicense) {
    cout << "Allowed to Drive" << endl;
}
```

SHORT-CIRCUIT EVALUATION

- In A && B, if A is false, B is NEVER evaluated.
- In A || B, if A is true, B is NEVER evaluated.
- Interview trick: use short-circuiting to prevent runtime crashes — if (ptr != nullptr && ptr->val == 10).

6. Nested if-else Statements

Definition: An if or else statement placed inside another if or else block.

Code Example: Leap Year Verification

C++

```
int year = 2024;
if (year % 4 == 0) {
    if (year % 100 == 0) {
        if (year % 400 == 0) cout << "Leap Year";
        else cout << "Not Leap Year";
    } else {
        cout << "Leap Year";
    }
} else {
    cout << "Not Leap Year";
}
```

Output: Leap Year

7. The switch Statement

Definition: A control statement that tests an integral or character expression against multiple constant values, called case labels.

Syntax

C++

```
switch (expression) {
    case value1:
        // Code
        break;
    case value2:
        // Code
        break;
    default:
        // Executed if no case matches
}
```

Example & Output

C++

```
char grade = 'B';
switch (grade) {
    case 'A': cout << "Excellent"; break;
    case 'B': cout << "Good"; break;
    default: cout << "Invalid";
}
```

Output: Good

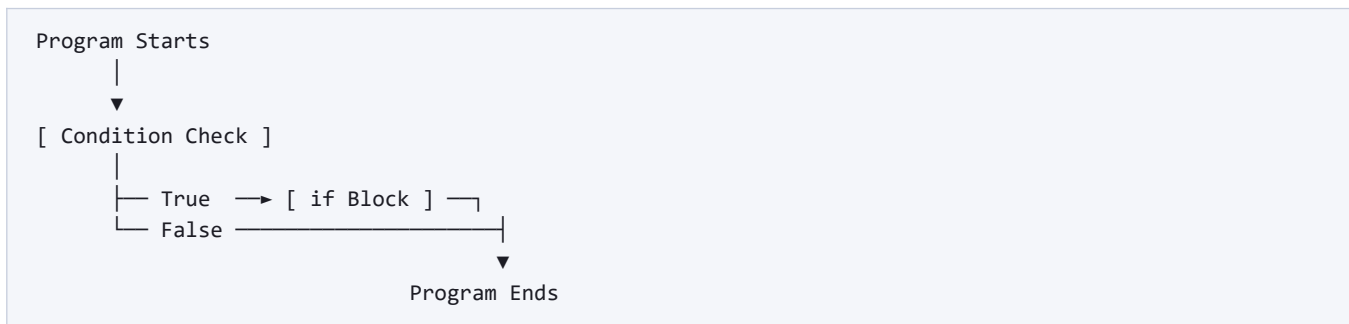
SECTION 3: Syntax & Keyword Breakdown

Keyword Reference

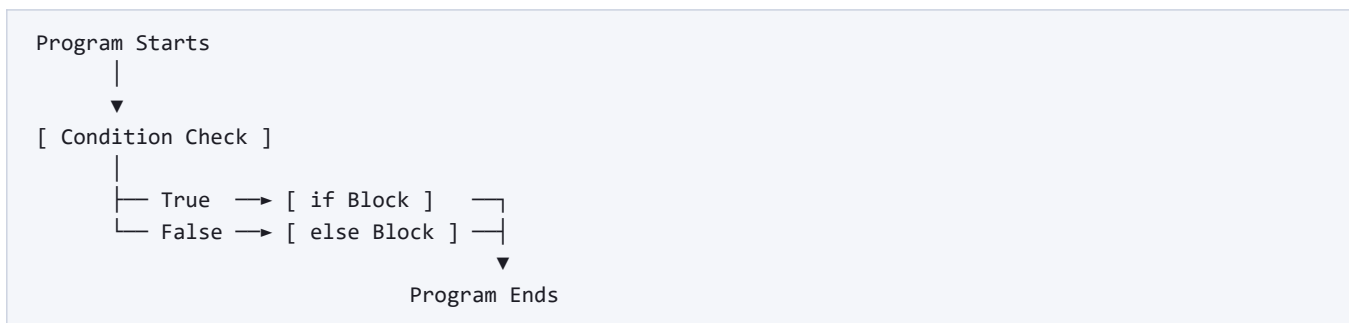
Keyword	Description & Usage	Constraints / Rules
if	Initiates conditional evaluation.	Condition must be enclosed in parentheses.
else	Defines the fallback block when the if condition fails.	Must immediately follow an if block.
switch	Multi-way decision dispatcher.	Expression MUST resolve to an integral, char, or enum type.
case	Label representing a prospective match value.	Value must be a compile-time constant. Duplicate cases are forbidden.
break	Exits the current switch or loop block.	If missing, execution falls through to the next case!
default	Catch-all handler when no case matches.	Optional, but best practice to include it at the end.

SECTION 4: Flow of Code Execution Diagrams

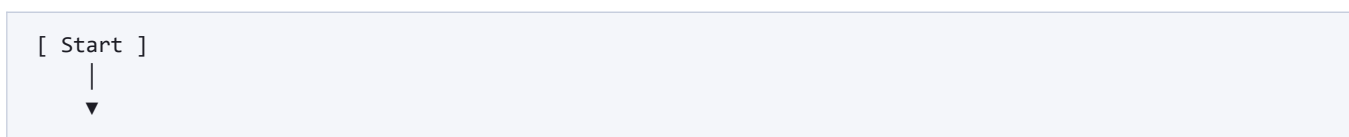
1. if Flow

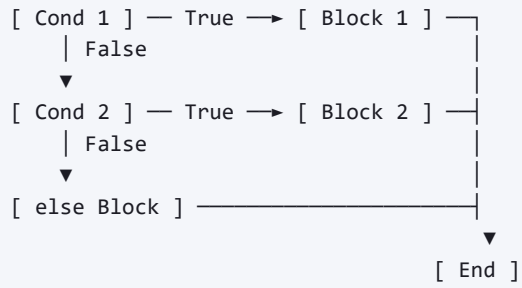


2. if-else Flow

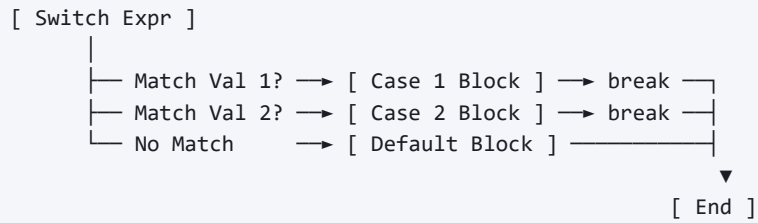


3. else-if Ladder Flow





4. switch Execution Flow



SECTION 5: Comprehensive Comparison Tables

1. if-else vs switch

if-else Ladder vs switch Statement

Feature	if-else Ladder	switch Statement
Expression Type	Evaluates ranges, relational, and logical expressions.	Evaluates discrete integers, characters, or enums only.
Performance	Linear search — $O(N)$ evaluation speed.	Optimized jump table / hash lookup — $O(1)$ execution speed.
Floating Point	Supported (e.g. if ($x > 3.14$))	NOT allowed (causes a compiler error).
Readability	Cluttered for numerous discrete scalar checks.	Clean, structured, and easy to maintain.
Use Case	Range checks (e.g. marks ≥ 80).	Menu-driven choices, discrete codes, keyboard input.

2. else-if vs Multiple if

else-if Ladder vs Multiple if Statements

Feature	else-if Ladder	Multiple if Statements
Execution	Stops evaluating after the first matching true condition.	Evaluates ALL conditions unconditionally.
Performance	Faster, due to early termination.	Slower, because every check executes.
Mutual Exclusivity	Guarantees that at most ONE block executes.	Multiple blocks can execute if conditions overlap.

SECTION 6: Operators & Short-Circuiting

1. Relational Operators

Relational Operators (a=10, b=20)

Operator	Name	Example	Result
>	Greater Than	$a > b$	false
<	Less Than	$a < b$	true
>=	Greater Than or Equal To	$a \geq 10$	true
<=	Less Than or Equal To	$b \leq 15$	false
==	Equal To	$a == b$	false
!=	Not Equal To	$a != b$	true

2. Logical Operators & Truth Table

Logical Operators

A	B	A && B	A B	!A
false	false	false	false	true
false	true	false	true	true
true	false	false	true	false
true	true	true	true	false

3. Operator Precedence (Highest → Lowest)

1. Parentheses ()
2. Logical NOT !
3. Relational >, <, >=, <=
4. Equality ==, !=
5. Logical AND &&
6. Logical OR ||

SECTION 7: In-Depth Switch Masterclass

Fall-Through Feature

If a case block does not end with `break`, execution continues into the subsequent cases regardless of their own condition.

```
C++
switch(x) {
    case 1: cout << "1"; // No break
    case 2: cout << "2"; break;
}
// If x = 1, Output will be "12"
```

Multiple Case Labels (Grouping)

```
C++
switch(ch) {
    case 'a': case 'e': case 'i': case 'o': case 'u':
        cout << "Vowel"; break;
    default:
        cout << "Consonant";
}
```

Data Types

Allowed: `int`, `char`, `short`, `long`, `enum`, `bool`.

Disallowed: `float`, `double`, `std::string`, and custom objects.

SECTION 8: Built-in Character & System Functions

Character & System Functions

Function	Header	Params	Returns	Description / Example
<code>isdigit(c)</code>	<code><cctype></code>	<code>char/int</code>	<code>int (bool)</code>	Non-zero if digit (0-9). <code>isdigit('5') → true</code>
<code>isalpha(c)</code>	<code><cctype></code>	<code>char/int</code>	<code>int (bool)</code>	Non-zero if letter (a-z, A-Z). <code>isalpha('A') → true</code>
<code>isalnum(c)</code>	<code><cctype></code>	<code>char/int</code>	<code>int (bool)</code>	Non-zero if letter or digit. <code>isalnum('#') → false</code>
<code>islower(c)</code>	<code><cctype></code>	<code>char/int</code>	<code>int (bool)</code>	Checks lowercase. <code>islower('a') → true</code>
<code>isupper(c)</code>	<code><cctype></code>	<code>char/int</code>	<code>int (bool)</code>	Checks uppercase. <code>isupper('Z') → true</code>

Function	Header	Params	Returns	Description / Example
tolower(c)	<cctype>	char/int	int (char)	Converts to lowercase. tolower('G') → 'g'
toupper(c)	<cctype>	char/int	int (char)	Converts to uppercase. toupper('m') → 'M'
isspace(c)	<cctype>	char/int	int (bool)	Checks whitespace/tab/newline. isspace(' ') → true
exit(code)	<cstdlib>	int code	void	Immediately terminates program execution. exit(0)

SECTION 9: Common Beginner Mistakes & Debugging

1. Assignment (=) vs Comparison (==)

INCORRECT

```
if (x = 5) { cout << "Equal"; } // Always assigns 5 & executes!
```

CORRECT

```
if (x == 5) { cout << "Equal"; }
```

2. Semicolon After Condition

INCORRECT

```
if (x > 10);
{ cout << "Greater"; } // Executes unconditionally – semicolon ends if prematurely!
```

CORRECT

```
if (x > 10) { cout << "Greater"; }
```

3. Missing break in switch

INCORRECT

```
switch(x) {
    case 1: cout << "One"; // Falls through to case 2!
    case 2: cout << "Two";
}
```

CORRECT

```
switch(x) {
    case 1: cout << "One"; break;
    case 2: cout << "Two"; break;
}
```

4. switch with Floating Point

INCORRECT

```
float val = 2.5;
switch(val) { case 2.5: cout << "OK"; } // Compilation Error!
```

CORRECT

```
if (val == 2.5) { cout << "OK"; } // Use if-else for float
```

X COMMON MISTAKE

- Accidental assignment inside a condition: `if (x = 5)` vs `if (x == 5)`.
- Semicolon right after a condition: `if (x > 10);` terminates the control block early.

- Missing break causes unexpected fall-through in switch.
- Using float or double variables inside switch cases.
- Dangling else ambiguity — else always matches the closest preceding unmatched if.

SECTION 10: Top Placement Interview Questions

Q1. Output Prediction (Short-Circuit Evaluation)

```
C++
int a = 0, b = 5;
if (a && ++b) {
    cout << "Inside Block";
}
cout << "b = " << b;
```

Answer: b = 5

Explanation: Because a is 0 (false), the logical AND (&&) short-circuits. The expression ++b is NEVER executed, so b remains 5.

Q2. Dangling Else Problem

```
C++
int x = 5, y = 0;
if (x > 2)
    if (y > 2)
        cout << "AA";
    else
        cout << "BB";
```

Answer: BB

Explanation: In C++, an else always pairs with the nearest preceding unclosed if. Here, else attaches to if (y > 2), not if (x > 2).

SECTION 11: Quick Revision Sheet

Important Syntax Summary

```
C++
if (cond) { ... }
if (cond) { ... } else { ... }
switch (val) { case 1: ... break; default: ... }
(cond) ? true_val : false_val; // Ternary
```

Short Tricks & Golden Rules

- Non-Zero Integer = true, Zero = false.
- && stops on the 1st false.
- || stops on the 1st true.
- switch only works with integral/char constants.

Decision Tree for Choosing Control Structures

Do you need to make a decision?

| (Yes)



How many options are available?

|
(Single)



Use [if]

|
(Two)



Use [if-else]

|
(Multiple)



Are conditions based on
discrete constants?

|
(Yes)



Use [switch]

|
(No)



Use [else-if ladder]

SECTION 12: One-Page Printable Cheat Sheet Summary

C++ Control Statements — Quick Recap

Construct	Best For	Key Constraint
if	Single conditional check	Body skipped if false
if-else	2-way branching	Mutually exclusive paths
else-if ladder	Range checking (e.g. marks > 90)	Evaluates top-to-bottom sequentially
switch	Menu choices / discrete matches	No floating points; compile-time constants

Top 5 Interview Pitfalls

1. Accidental assignment inside a condition: if (x = 5) vs if (x == 5).
2. Semicolon right after a condition: if (x > 10); terminates the control block.
3. Missing break statement causes unexpected fall-through in switch.
4. Attempting to use float or double variables in switch cases.
5. Dangling else ambiguity — else matches the closest preceding unmatched if.

End of C++ Control Statements Cheat Sheet

NextGen Data Minds